

L02- Deploying an AI Model on a Simulated Edge Device 2 Options

Martin Demel, Maira Tanweer Chachar, Jenner Ramos, Ryan Yaunch

Department of Science, Technology, Engineering & Math, Houston Community
College

ITAI-3377-A.I. at the Edge & IIOT Env-RT-17181

Professor Sitaram Ayyagari

January 28, 2026.

Introduction

In this lab, our team set up a Python environment, trained a simple neural network on the MNIST dataset, converted the trained model to TensorFlow Lite format, and validated inference using the TensorFlow Lite Interpreter. The goal was to practice the end-to-end workflow for moving a trained model from development into a lightweight format suitable for deployment on devices with limited resources.

Body

Part 1: Setting Up the Development Environment

Step 1: Verify Python and TensorFlow Installation

We began by confirming the Python version in our environment and verifying the installed TensorFlow package. This step ensured our team was working with a compatible setup before starting model training and conversion.

```
Python 3.10.16
Name: tensorflow
Version: 2.15.1
Summary: TensorFlow is an open source machine learning framework for everyone.
Home-page: https://www.tensorflow.org/
Author: Google Inc.
Author-email: packages@tensorflow.org
License: Apache 2.0
Location: /Users/martin.demel/myenv3.10/lib/python3.10/site-packages
Requires: absl-py, astunparse, flatbuffers, gast, google-pasta, grpcio, h5py,
keras, libclang, ml-dtypes, numpy, opt-einsum, packaging, protobuf, setuptools,
six, tensorboard, tensorflow-estimator, tensorflow-io-gcs-filesystem, termcolor,
typing-extensions, wrapt
Required-by: tensorflow-macos, tf_keras
```

Screenshot 1. Python and TensorFlow version output

Step 2: Install TensorFlow and Jupyter Notebook

Next, we installed the required packages. TensorFlow provided the training and conversion tools, and Jupyter Notebook provided the interface for running the lab steps in order and for reviewing outputs as we progressed.

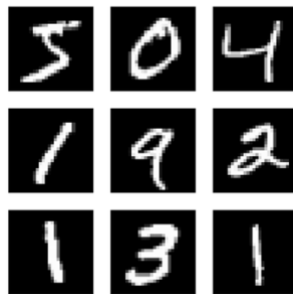
```
Requirement already satisfied: tensorflow==2.15.1 in
/Users/martin.demel/myenv3.10/lib/python3.10/site-packages (2.15.1)
Requirement already satisfied: jupyter in /Users/martin.demel/myenv3.10/lib/python3.10/site-
packages (1.1.1)
Requirement already satisfied: notebook in /Users/martin.demel/myenv3.10/lib/python3.10/site-
packages (from jupyter) (7.4.5)
Requirement already satisfied: jupyter-console in
/Users/martin.demel/myenv3.10/lib/python3.10/site-packages (from jupyter) (6.6.3)
Requirement already satisfied: nbconvert in /Users/martin.demel/myenv3.10/lib/python3.10/site-
packages (from jupyter) (7.16.6)
```

Screenshot 2. Package installation confirmation for TensorFlow and Jupyter

Part 2: Creating and Training an AI Model

Step 3: Load the MNIST Dataset

We loaded the MNIST dataset using the Keras dataset loader and normalized the pixel values to the range [0, 1]. To confirm that the dataset loaded correctly, we displayed a small grid of sample handwritten digit images.



Screenshot 3. Sample MNIST images displayed in a 3 by 3 grid

Step 4: Define and Train a Neural Network

After validating the dataset, we built a simple feedforward neural network with Keras. We trained the model for five epochs, monitored accuracy during training, evaluated performance on the test set, and saved the trained model to the file `mnist_model.h5` for conversion in the next section.

```

Epoch 3/5
1875/1875 [=====] - 13s 7ms/step - loss: 0.3161 - accuracy: 0.9142 - val_loss: 0.3154
- val_accuracy: 0.9145
Epoch 4/5
1875/1875 [=====] - 13s 7ms/step - loss: 0.3284 - accuracy: 0.9114 - val_loss: 0.3138
- val_accuracy: 0.9162
Epoch 5/5
1875/1875 [=====] - 13s 7ms/step - loss: 0.3391 - accuracy: 0.9108 - val_loss: 0.3635
- val_accuracy: 0.9067
Test accuracy: 0.9067
Model training complete and saved as mnist_model.h5

```

Screenshot 4. Training logs and saved model confirmation

Part 3: Converting and Saving the Model

Step 5: Convert the Model to TensorFlow Lite Format

With the trained model saved, we loaded `mnist_model.h5` and ran the TensorFlow Lite converter to generate a compact model file. We then saved the converted output as `mnist_model.tflite`, a format commonly used for on-device inference.

```

INFO:tensorflow:Assets written to: /var/folders/m/nk8g4cs5tz8t1yj2kgl8p80000gn/T/tmpk1fw6gbx/assets
INFO:tensorflow:Assets written to: /var/folders/m/nk8g4cs5tz8t1yj2kgl8p80000gn/T/tmpk1fw6gbx/assets
Model successfully converted and saved as mnist_model.tflite
2026-01-28 14:26:10.743501: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378] Ignored output_format.
2026-01-28 14:26:10.743520: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381] Ignored drop_control_dependency.
2026-01-28 14:26:10.743696: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedModel from: /var/folders/m/nk8g4cs5tz8t1yj2kgl8p80000gn/T/tmpk1fw6gbx
2026-01-28 14:26:10.744114: I tensorflow/cc/saved_model/reader.cc:51] Reading meta graph with tags { serve }
2026-01-28 14:26:10.744118: I tensorflow/cc/saved_model/reader.cc:146] Reading SavedModel debug info (if present) from: /var/folders/m/nk8g4cs5tz8t1yj2kgl8p80000gn/T/tmpk1fw6gbx
2026-01-28 14:26:10.744945: I tensorflow/compiler/mlir/mlir_graph_optimization_pass.cc:388] MLIR V1 optimization pass is not enabled
2026-01-28 14:26:10.745257: I tensorflow/cc/saved_model/loader.cc:233] Restoring SavedModel bundle.
2026-01-28 14:26:10.760283: I tensorflow/cc/saved_model/loader.cc:217] Running initialization op on SavedModel bundle at path: /var/folders/m/nk8g4cs5tz8t1yj2kgl8p80000gn/T/tmpk1fw6gbx
2026-01-28 14:26:10.764512: I tensorflow/cc/saved_model/loader.cc:316] SavedModel load for tags { serve }; Status: success: OK. Took 20816 microseconds.
2026-01-28 14:26:10.769794: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269] disabling MLIR crash reproducer, set env var 'MLIR_CRASH_REPRODUCER_DIRECTORY' to enable.

```

Screenshot 5. Successful conversion message for the TensorFlow Lite model

Part 4: Loading and Running Inference with TensorFlow Lite

Step 6: Load the Converted Model Using the TensorFlow Lite Interpreter

Next, we created a TensorFlow Lite Interpreter instance pointing to `mnist_model.tflite` and allocated tensors. We printed the input and output tensor details to confirm that the model loaded correctly and that the expected shapes matched an MNIST input image and a ten-class output.

```

Input Details: [{'name': 'serving_default_flatten_input:0', 'index': 0, 'shape': array([ 1, 28, 28],
dtype=int32), 'shape_signature': array([-1, 28, 28], dtype=int32), 'dtype': <class
'numpy.float32>', 'quantization': (0.0, 0), 'quantization_parameters': {'scales': array([],
dtype=float32), 'zero_points': array([], dtype=int32), 'quantized_dimension': 0},
'sparsity_parameters': {}}]
Output Details: [{'name': 'StatefulPartitionedCall:0', 'index': 9, 'shape': array([ 1, 10],
dtype=int32), 'shape_signature': array([-1, 10], dtype=int32), 'dtype': <class 'numpy.float32>',
'quantization': (0.0, 0), 'quantization_parameters': {'scales': array([], dtype=float32),
'zero_points': array([], dtype=int32), 'quantized_dimension': 0}, 'sparsity_parameters': {}}]
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.

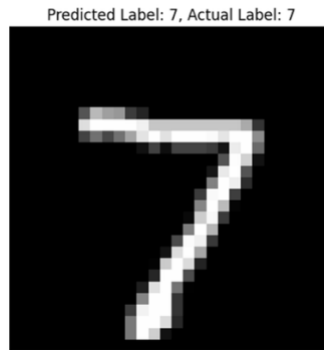
```

Screenshot 6. Interpreter input and output tensor details

Step 7: Perform Inference with TensorFlow Lite

Finally, we selected one test image, formatted it to match the interpreter's input, and ran inference.

We displayed the test image and compared the predicted label with the true label to verify that the converted model produced the correct result.



Screenshot 7. Inference result showing predicted and actual labels

Conclusion

By completing this notebook, our team demonstrated the end-to-end workflow from training a neural network to converting it to TensorFlow Lite format and running inference with the TensorFlow Lite Interpreter. This process mirrors what is required to prepare models for deployment in resource-constrained environments, where smaller model size and efficient execution are important for real-world applications.